

JVMud User Manual

JVMud Project

Version 0.1.0-SNAPSHOT

Table of Contents

Overview	1
Who This Manual Is For	1
Project Status	1
Installing	2
Prerequisites	2
Repository Layout	2
Build And Test	2
Build This Manual	3
Running JVMud	4
Start The Admin CLI	4
Start The Telnet Server	4
Connect As A Player	4
Run The Startup Smoke Test	5
Admin CLI	6
Starting The Shell	6
Common Commands	6
Mudlib Filesystem	7
Verbosity	7
Playing	8
Login	8
Basic Commands	8
How World Input Is Routed	8
Current Limits	8
Mudlib Layout	9
Important Paths	9
Upstream Source Policy	9
Engine And Mudlib Boundary	9
LPC Authoring	10
Explicit Types	10
Compatibility Shims	10
Compile And Load Checks	10
Java API Docs	10
Troubleshooting	11
Build Fails	11
Object Load Fails	11
Player Login Or Commands Fail	11
Mudlib Compatibility Report	11
Manual Generation Fails	12

Overview

JVMud is an experimental LPC/LPMud text-world engine for the JVM. It compiles LPC-authored mudlib objects into JVM classes, hosts them inside a Java runtime, and exposes both an admin shell and a player-facing Telnet server.



The project is intentionally split into a few clear layers:

- The runtime owns JVMud-native world concepts such as worlds, places, links, entities, locations, capabilities, containment, and scheduling.
- The compiler owns LPC scanning, preprocessing, parsing, semantic analysis, intermediate representation, bytecode generation, efun contracts, and host-facing object loading.
- The server hosts a mudlib as a shared interactive world and accepts Telnet player sessions.
- The admin CLI gives a local operator a direct shell for loading, inspecting, cloning, moving, calling, and reloading objects.
- The bundled `mudlibs/lp245` tree is the vanilla LPMUD 2.4.5 compatibility target.

JVMud uses the human-facing chain `Player` → `Session` → `Persona` → `World`. A player connects through a session. The session attaches to a persona. The persona is present in the world. The term `player object` is reserved for the mudlib-side LPC object used by LP245 compatibility.

Who This Manual Is For

This manual is for people who want to run JVMud, inspect the bundled mudlib, connect as a player, or author LPC content against the current project shape. Java API details live in the generated API docs; this manual explains the user and operator path first.

Project Status

JVMud is early-stage engine work. The current repository can build with Maven, run compiler and runtime tests, start an admin shell, and start a Telnet-capable server against the bundled LP245 mudlib. Compatibility is deliberately incremental, so some vanilla mudlib paths may expose compiler, runtime, or shim gaps.

Installing

JVMud is built from the repository root with Maven and Java.

Prerequisites

Install:

- Java 21 or newer.
- Maven.
- A terminal that can run the repository shell launchers.

The repository includes launcher scripts for the admin CLI and Telnet server. Those scripts compile the required Maven modules before starting the selected entrypoint.

Repository Layout

Path	Purpose
<code>runtime/</code>	JVMud engine runtime source.
<code>compiler/</code>	LPC compiler, generated-code helpers, efun contracts, and host-facing runtime loader APIs.
<code>server/</code>	Player-facing server source, including the Telnet entrypoint.
<code>cli/</code>	Local admin shell source.
<code>mudlibs/lp245/</code>	Bundled vanilla LPMUD 2.4.5 mudlib and JVMud compatibility shims.
<code>docs/</code>	Static project website, generated API docs, and generated manual output.
<code>manual/</code>	Source files for this user manual.

Build And Test

From the repository root, run:

```
mvn test
```

The test suite includes compiler/runtime smoke coverage and an informational mudlib compatibility scan. The scan writes:

```
compiler/target/jvmud-mudlib-compatibility.md
```

That report is a readiness signal, not a failing test gate.

Build This Manual

The manual is generated from AsciiDoc source:

```
mvn -Pmanual-docs process-resources
```

The generated outputs are:

- [docs/manual/index.html](#)
- [docs/manual/jvmud-user-manual.pdf](#)

Running JVMud

JVMud currently has two main local entrypoints: the admin CLI and the player-facing Telnet server.

Start The Admin CLI

Run:

```
./jvmud-admin
```

Optionally pass a mudlib root as the first argument:

```
./jvmud-admin mudlibs/lp245
```

The admin CLI is a single-user operator shell. It is useful for loading objects, inspecting compiled state, invoking methods, walking the mudlib filesystem, and diagnosing compile or load failures.

Start The Telnet Server

Run:

```
./jvmud-start
```

The default server uses:

- mudlib root: `mudlibs/lp245`
- host: `localhost`
- port: `4000`
- config path: `jvmud/config`

You can override those defaults:

```
./jvmud-start -mudlib-dir mudlibs/lp245 -host localhost -port 4000 -config  
jvmud/config
```

Starting the server boots one shared runtime and world. Each Telnet connection attempts to attach the configured mudlib player object and uses a host-owned fallback persona only if that path fails.

Connect As A Player

In another terminal, connect with a Telnet client:

```
telnet localhost 4000
```

Use `/help` for session controls and `/quit` to disconnect. Admin inspection and object mutation stay in the admin CLI.

Run The Startup Smoke Test

The real entrypoint smoke test launches `jvmud-start`, connects over TCP, completes the mudlib login flow when needed, and verifies basic player commands:

```
scripts/smoke-jvmud-start.sh
```

Use this check after meaningful changes to startup, login, player attachment, command dispatch, or mudlib compatibility behavior.

Admin CLI

The admin CLI is a local shell for operator and development work. Every input line is parsed as an admin command.

Starting The Shell

Run:

```
./jvmud-admin
```

The launcher compiles the CLI and compiler modules, then starts the shell with local build output.

Common Commands

The command surface includes:

- `boot`
- `call`
- `cat`
- `cd`
- `clone`
- `destruct`
- `inspect`
- `load`
- `look`
- `ls`
- `move`
- `objects`
- `pwd`
- `reload`
- `verbosity`
- `where`
- `quit`

Run `help` inside the shell for the current command list, aliases, and usage details.

Mudlib Filesystem

The shell exposes a mudlib-rooted virtual filesystem. If the mudlib root is `/Users/jonathan/Projects/jvmud/mudlibs/lp245`, then CLI path `/` maps to that real directory. Filesystem commands cannot navigate above the configured mudlib root.

Verbosity

Use:

```
verbosity quiet  
verbosity normal  
verbosity watch
```

`watch` prints compiler stage progress for commands such as `load` and `clone`. It is the quickest way to separate parser, analyzer, lowering, bytecode, reset, and load failures while diagnosing mudlib objects.

Playing

The player path runs through the Telnet server, not the admin CLI.

Login

After connecting to the Telnet server, JVMud attempts to attach the configured LP245 player object. New players may be prompted for creation details such as name, password confirmation, email, or gender, depending on the current mudlib state. Returning players with saved state should skip character creation prompts.

Basic Commands

Useful early commands include:

- `look`
- movement commands such as `north`
- `get all`
- `light stick`
- `help`

Session controls use slash commands:

- `/help`
- `/quit`

Slash commands are limited to session control. World interaction is routed through the mudlib action system.

How World Input Is Routed

Player input is routed through LPC `init`, `add_action`, and `add_verb` registrations on nearby or carried objects. This keeps player commands mudlib-facing while the engine owns the live world, session, persona, containment, movement, and scheduling model.

Current Limits

The player-facing server is still early development software. Session-to-session messaging, output isolation between participants, and production networking policy belong to later server work.

Mudlib Layout

The bundled mudlib lives at:

```
mudlibs/lp245/
```

Treat that tree as LPC source and content, not Java module source.

Important Paths

Path	Purpose
<code>mudlibs/lp245/jvmud/config</code>	JVMud mudlib configuration, including lifecycle and compatibility paths.
<code>mudlibs/lp245/jvmud/mudlib.c</code>	Mudlib-side boundary shim object.
<code>mudlibs/lp245/jvmud/mfun.c</code>	Mudlib-side compatibility functions used as a simul-efun style support layer.
<code>mudlibs/lp245/obj/</code>	Reusable LPC object definitions, including player-oriented objects.
<code>mudlibs/lp245/room/</code>	World and room content.

Upstream Source Policy

Preserve upstream vanilla mudlib files by default. Compatibility belongs in dedicated JVMud shim objects and focused compiler/runtime support unless a task explicitly asks for mudlib style or formatting changes.

Engine And Mudlib Boundary

The engine owns JVMud-native concepts: world, place, link, entity, location, capability, containment, presence, persistence, and time. The LP245 mudlib can keep legacy vocabulary and behavior behind compatibility shims, but the engine should not become a copy of the old driver ontology.

LPC Authoring

JVMud compiles LPC source to JVM classes and loads the resulting objects into the runtime.

Explicit Types

JVMud does not accept untyped LPC method declarations or untyped method parameters. Add explicit LPC return and parameter types when mudlib source needs to compile.

```
void reset(int arg) {  
    if (arg) {  
        return;  
    }  
}
```

Compatibility Shims

When legacy LP245 behavior needs JVMud support, prefer one of these paths:

- Add or update a dedicated mudlib-side shim under `mudlibs/lp245/jvmud/`.
- Add focused compiler/runtime support for a real compatibility gap.
- Keep the engine API named for JVMud concepts rather than legacy LPC driver concepts.

Compile And Load Checks

Use the admin CLI to compile or load focused objects:

```
load /room/village/vill_green  
inspect /room/village/vill_green
```

Use `verbosity watch` when you need to see which compiler or runtime stage failed.

Java API Docs

Generated Java API docs are published separately under `docs/api`. Use them when authoring or diagnosing code that crosses into Java runtime, compiler, efun, or host-facing loader APIs.

Troubleshooting

JVMud issues are easiest to diagnose when compile, load, boot, and live player behavior are kept separate.

Build Fails

Run:

```
mvn test
```

If dependencies are missing for launcher scripts, run the baseline Maven build once so local dependencies are resolved.

Object Load Fails

In the admin CLI:

```
verbosity watch  
load /path/to/object
```

Read the failing stage first. Parser, semantic analysis, bytecode generation, direct object load, reset-time behavior, and command dispatch failures often need different fixes.

Player Login Or Commands Fail

Run the real startup smoke:

```
scripts/smoke-jvmud-start.sh
```

This checks the configured player object path, login prompts, movement, and representative LP245 commands through the same process boundary a player uses.

Mudlib Compatibility Report

The compatibility report is written to:

```
compiler/target/jvmud-mudlib-compatibility.md
```

Use it as a broad readiness signal. For a user-visible bug, prefer a focused admin CLI load or live Telnet smoke that reproduces the behavior.

Manual Generation Fails

Regenerate the manual with:

```
mvn -Pmanual-docs process-resources
```

The HTML and PDF are generated from the same `manual/index.adoc` include tree. Do not edit generated files under `docs/manual/` as the source of truth.